

Lambda expressions

In Python, a **lambda function** is a small, **anonymous function** defined using the `lambda` keyword. It can take any number of arguments but is restricted to a **single expression**, whose result is returned automatically. Unlike functions created with `def`, lambdas are typically used for short, throwaway operations.

Example:

```
square = lambda x: x ** 2

print(square(5)) # Output: 25
```

Here, `square` is a lambda that returns the square of its input.

Common Use Cases:

- **Inline Conditional Logic**

```
check = lambda x: "Positive" if x > 0 else "Negative" if x < 0 else "Zero"

print(check(-3)) # Output: Negative
```

- **With `map()` for transformations**

```
nums = [1, 2, 3, 4]

doubled = list(map(lambda x: x * 2, nums))

print(doubled) # Output: [2, 4, 6, 8]
```

- **With `filter()` for selection**

```
nums = [1, 2, 3, 4, 5, 6]

evens = list(filter(lambda x: x % 2 == 0, nums))

print(evens) # Output: [2, 4, 6]
```

- **With `reduce()` for aggregation**

```
from functools import reduce

nums = [1, 2, 3, 4]

product = reduce(lambda x, y: x * y, nums)

print(product) # Output: 24
```

- **As a key in sorting**

```
students = [("Alice", 25), ("Bob", 20), ("Charlie", 23)]

sorted_students = sorted(students, key=lambda x: x[1])

print(sorted_students) # Output: [('Bob', 20), ('Charlie', 23),
('Alice', 25)]
```

Key Differences from def:

- **Lambda:** Single expression, no name unless assigned, no docstring, best for short-lived logic.
- **def:** Multiple statements, named, supports docstrings, better for reusable/complex logic.

Global vs Local Scope

In Python, the **scope** of a variable determines where it can be accessed or modified in the program. Variables can either have a **global scope** or a **local scope**, depending on where they are defined.

Local Scope

A variable has a **local scope** when it is defined inside a function. Such variables are accessible only within the function and are destroyed once the function execution is complete.

Example:

```
def greet():

    msg = "Hello from inside the function!" # Local variable

    print(msg)

greet()

print(msg) # This will raise a NameError
```

Explanation: The variable `msg` exists only within the `greet()` function. Attempting to access it outside the function results in an error.

Global Scope

A variable has a **global scope** when it is defined outside any function. It can be accessed and used anywhere in the program, including inside functions.

Example:

```
msg = "Python is awesome!" # Global variable

def display():
    print("Inside function:", msg)

display()

print("Outside function:", msg)
```

Explanation: The variable `msg` is accessible both inside and outside the `display()` function because it is declared globally.

Shadowing and Modifying Global Variables

If a local variable inside a function has the same name as a global variable, the local variable **shadows** the global one within the function. To modify a global variable inside a function, the `global` keyword must be used.

Example of shadowing:

```
x = 10 # Global variable

def func():
    x = 5 # Local variable shadows the global one
    print("Inside function:", x)

func()

print("Outside function:", x)
```

Output:

Inside function: 5

Outside function: 10

Example of modifying a global variable:

```
x = 10 # Global variable

def func():

    global x

    x += 5 # Modifies the global variable

    print("Inside function:", x)

func()

print("Outside function:", x)
```

Output:

```
Inside function: 15

Outside function: 15
```

Key Differences Between Global and Local Scope

- **Definition:** Global variables are declared outside functions, while local variables are declared inside functions.
- **Lifetime:** Global variables exist throughout the program's execution, whereas local variables exist only during the function's execution.
- **Accessibility:** Global variables can be accessed anywhere, while local variables are restricted to their function.
- **Modification:** Local variables do not affect global variables unless explicitly declared as `global`.

Understanding the distinction between global and local scope is crucial for writing clean, maintainable Python code.